

C 语言课程设计报告

课程设计选题： 农田无人机喷洒农药模拟系统

小组成员姓名： 刘俊辉 吴天宇

小组成员学号： U202514535 U202514546

小组成员班级： 人工智能 2503 班

正式验收时间： 2026 年 6 月 7 日

一、 课题需求分析

我们组的课题为“农田无人机喷洒农药模拟系统”，我们对其的定位与开发方向为运行在嵌入式平台上的农场经营仿真游戏。游戏围绕“种植—生长—病虫害—无人机防治—收获—交易—升级”的核心玩法展开，目标是实现完整且可交互的农业经营流程以及无人机侦察与喷洒农药的模拟。

在游戏中，玩家扮演农场主，经营农场、种植作物，面对随机发生的病虫害，需要操控无人机进行勘察、配药与自动喷洒，最终收获作物、赚取金币、提升等级，体验完整的农业生产管理流程。

二、 课题总体设计方案

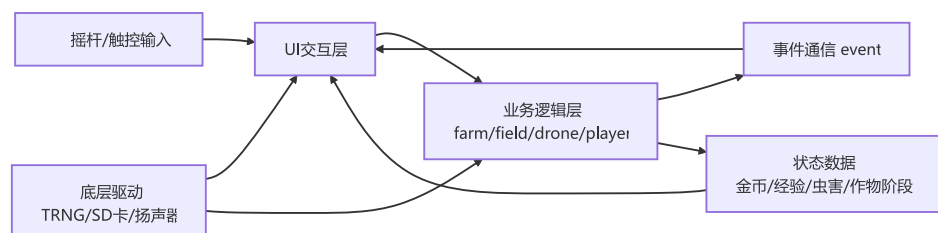
2.1 系统架构设计

系统采用“硬件驱动层 → 核心调度层 → 业务逻辑层 → 事件通信层 → UI 表现层”的五层分层架构，各层职责明确、耦合度低。

分层设计原则：

- 硬件驱动层封装所有底层硬件访问，向上提供统一接口。
- 核心调度层负责系统启动初始化、主循环与周期性心跳。
- 业务逻辑层实现游戏核心规则，不依赖任何 UI 代码。
- UI 表现层基于 LVGL 构建，仅通过事件系统与逻辑层通信，获取状态变更。
- 事件通信层向 UI 层通知状态变更，解耦逻辑层与 UI 层，降低模块耦合。

2.2 数据与控制流



游戏的 main 函数流程如下：

1. 硬件初始化（摇杆、扬声器、TRNG、SD 卡等）。
2. 创建业务对象单例（farm、player、drone）。
3. 初始化 LVGL 图形库与 UI 界面。
4. 启动心跳定时器（1 秒周期），驱动游戏时间推进。
5. 进入主循环：speaker_update() → lv_timer_handler() → 消费事件队列 → delay_us(2000)。

2.3 模块设计

系统共设计六个核心业务模块：

模块	文件	职责
farm	farm.c/h	农场容器，管理所有田块的集合、农场规模升级、存档读写
field	field.c/h	管理单个田块：种植、生长、虫害感染/清除、死亡判定、收获、升级
drone	drone.c/h	管理无人机状态：空闲/侦查/喷洒切换、路径规划、药仓管理、虫害检测、农药喷洒
player	player.c/h	玩家经济与成长：金币/经验/等级/折扣、购买/种植/收获/售卖、升级统一入口
event	event.c/h	环形事件队列：17 种事件类型，通知田地/无人机等相关状态变化，解耦业务层与 UI 层
data	data.c/h	游戏配置常量：价格表、经验表、升级参数、虫害模型参数等

此外，UI 层按功能细分为以下子模块：

子模块	路径	职责
ui	ui/ui.c/h	UI 初始化、界面切换、事件分发入口
ui_common	ui/ui_common.c/h	UI 公共控件工具函数

layouts	ui/layouts/	各页面布局实现（主界面、商店、仓库、设置、无人机、加载页）
callbacks	ui/callbacks/	各页面按钮回调函数
widgets	ui/widgets/	可复用组件（窗口、网格列表、消息提示）
icon	ui/icon/	图片资源 C 数组
audio	ui/audio.c/h	背景音乐与音效管理

三、 课题详细设计与实现过程

3.1 农场与田块模块（farm / field）

3.1.1 数据结构设计

农场采用单例模式，全局唯一，内部维护一个二维指针数组管理所有田块：

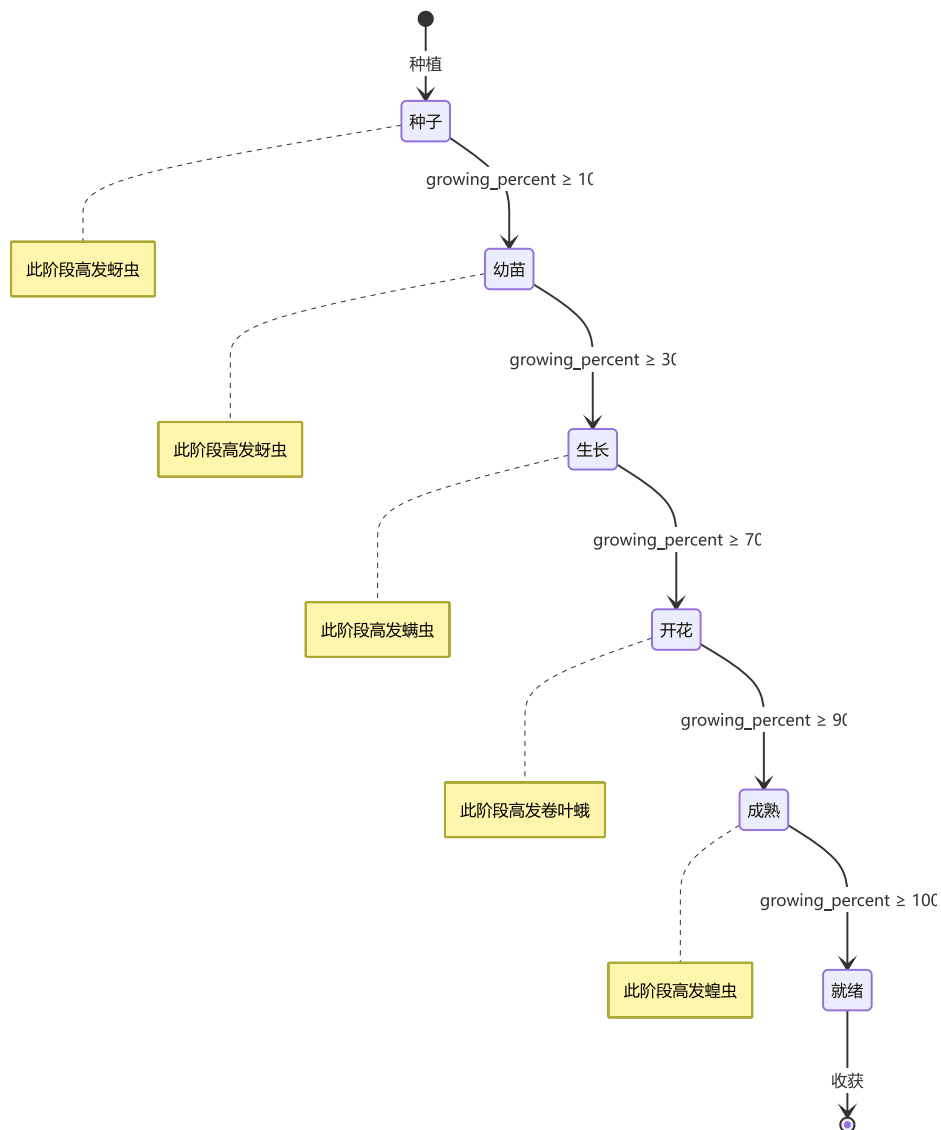
```
typedef struct {
    field_t *fields[GAME_GRID_SIZE][GAME_GRID_SIZE];
    int current_size;    // 当前可用田块边长 (4/5/6/7)
    int size_level;      // 农场规模等级 (0/1/2/3)
} farm_t;
```

田块结构体包含作物状态、虫害状态、升级等级与产量因子等完整信息：

```
typedef struct {
    int x, y;                // 网格坐标
    int output_level;        // 产量升级等级
    int ready_time_level;    // 生长速度升级等级
    int tolerance_level;     // 耐虫性升级等级
    crop_type_t crop_type;   // 作物种类
    int ready_time;          // 总成熟所需时间
    int growing_time;        // 已生长时间
    double growing_percent;  // 生长进度 (0.0~1.0)
    crop_stage_t stage;      // 生长阶段
    crop_damage_t damage;    // 虫害类型
    bool is_detected;        // 虫害类型是否已被无人机检测
    int base_output;         // 基础产量
    double factor;           // 产量因子 (受虫害影响衰减)
    double tolerance;        // 耐虫性 (升级提供)
} field_t;
```

3.1.2 作物生长阶段

作物经历六个生长阶段，由 `growing_percent` 字段驱动阶段转换：



`field_grow()` 函数由心跳定时器每秒调用一次，遍历所有已种植田块，推进生长时间、更新阶段、处理虫害逻辑。

3.1.3 虫害感染模型

每种虫害拥有独立的二次函数感染概率参量 (a, b, c) ，感染概率随生长进度 t 变化：

$$P_{\text{Infection}}(t) = \max(0, \min(1, a \cdot (t - b)^2 + c - \text{tolerance})) \cdot (1 - \text{tolerance})$$

其中 a 控制曲线开口与陡峭度， b 为峰值位置（对应高发生长阶段）， c 为峰值高度， tolerance 为田块耐虫性升级提供的抗性。

各虫害参数配置如下：

虫害	<i>a</i>	<i>b</i>	<i>c</i>	高发阶段	对应农药
蚜虫	-6	0.20	0.95	幼苗期	杀蚜剂
螨虫	-5	0.50	0.98	生长期	杀螨剂
卷叶蛾	-4	0.70	0.96	开花期	杀卷叶蛾剂
蝗虫	-3	0.85	0.90	成熟期	杀蝗剂

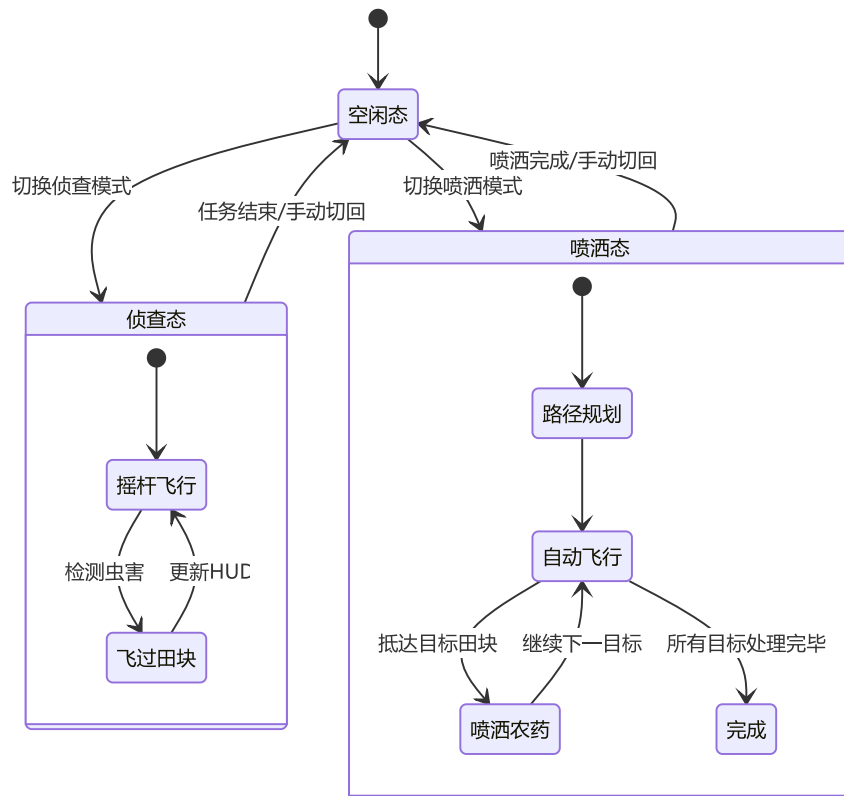
每个游戏刻作物生长时间+1，同时按概率判断是否命中虫害，若命中，则作物感染虫害。

虫害感染后，每游戏刻作物产量因子 *factor* 按 $\text{field_damage_decline_rate} \times (1 - \text{tolerance})$ 递减；若降至阈值 0.3 以下，则作物死亡。无虫害时因子按 *field_damage_recovery_rate* 缓慢恢复/增长（0.005/刻）。

3.2 无人机模块（drone）

3.2.1 无人机状态

无人机有三种工作状态：



- **空闲态**：无人机停留在农田外待命。
- **侦查态**：摇杆手动飞行，飞过田块时自动检测虫害类型，在 HUD 面板实时反馈各虫害数量。
- **喷洒态**：按路径规划算法自动飞行，对已侦查田块逐一喷洒匹配农药，在 HUD 面板实时更新各虫害数量与药仓农药余量。

无人机维护一个 `one_zero_matrix` 矩阵记录检测到的虫害分布，该矩阵可能与实际虫害情况存在信息差——因为田块总是要在无人机侦察后才能得知虫害情况。

3.2.2 路径规划算法

喷洒路径采用**贪心 + 2-opt 局部优化**算法，分为两个阶段：

第一阶段——最近邻贪心构造初始路径：从起点 (0,0) 出发，每次选择距离当前位置曼哈顿距离最近的未访问病田作为下一目标，逐步构造出一条经过所有病田的访问序列。时间复杂度 $O(n^2)$ ，其中 n 为病田数量。

第二阶段——2-opt 局部优化：在贪心初始路径基础上，反复检查路径中任意两条不相邻的边 $(i, i + 1)$ 和 $(j, j + 1)$ ，判断交换为 (i, j) 和 $(i + 1, j + 1)$ 后总路径长度是否缩短。若缩短则执行交换（反转 $i + 1$ 到 j 之间的访问顺序），直至无法进一步优化。2-opt 是 TSP 问题的经典局部搜索方法，能有效消除路径交叉，显著缩短总行程。



该算法兼顾了构造效率与路径质量：贪心阶段保证快速得到一个可行解，2-opt 阶段在不增加额外内存开销的前提下对路径进行精炼，适合嵌入式平台的计算资源约束。

3.2.3 药仓与匹配机制

无人机拥有独立药仓（容量 10~50，可升级），执行喷洒前需从玩家背包装载对应农药。喷洒时系统自动根据检测到的虫害类型匹配农药：

虫害	所需农药
蚜虫	杀蚜剂
螨虫	杀螨剂

卷叶蛾	杀卷叶蛾剂
蝗虫	杀蝗剂

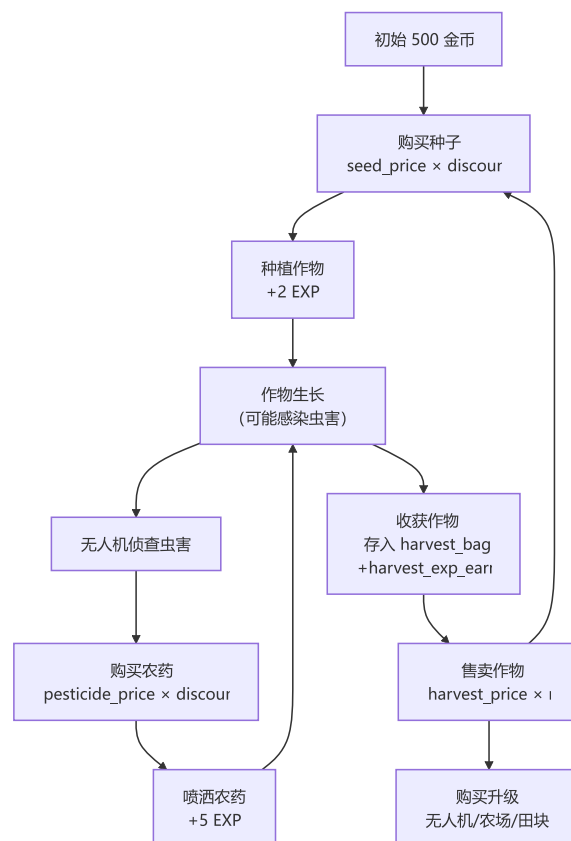
药量不足时无法对该田块喷洒，需返回后手动补药。喷洒完成后田块虫害清除，作物恢复正常生长，玩家获得 +5 经验。

3.3 玩家与经济模块（player）

3.3.1 数据结构

```
typedef struct {
    int experience;           // 当前经验值
    int level;                // 当前等级 (0~39)
    int level_stage;          // 等级段 (0~6, 对应折扣率)
    int coins;                // 金币数
    int seed_bag[CROP_TYPE_NONE]; // 种子背包
    int harvest_bag[CROP_TYPE_NONE]; // 收获背包
    int pesticide_bag[CROP_PESTICIDE_NONE]; // 农药背包
} player_t;
```

3.3.2 经济系统流程



3.3.3 经验与等级系统

经验获取途径：

操作	经验值
种植作物	+2 EXP
收获小麦	+2 EXP/单位产量
收获水稻	+3 EXP/单位产量
收获玉米	+4 EXP/单位产量
喷洒农药	+5 EXP

共 40 个等级，升级所需经验逐级递增（从 10 到 2210）。等级分为 6 个阶段，阶段阈值分别为 Lv.10、15、20、25、30、40，对应不同的商店折扣率：

等级段	折扣率
Lv.1-9	原价 (100%)
Lv.10-14	9.5 折
Lv.15-19	9 折
Lv.20-24	8.5 折
Lv.25-29	8 折
Lv.30-39	7.5 折
Lv.40	7 折

3.3.4 升级体系

系统提供 6 大升级维度，所有购买通过 `player_buy_*` 统一入口，自动校验等级折扣与金币余额：

类别	升级项	等级	价格梯度	效果
 无人机	飞行速度	0→3	1000/2500/5000	速度 10→15→20→30
 无人机	药仓容量	0→3	1000/2500/5000	容量 10→20→30→50
 农场	规模	0→3	5000/10000/20000	大小 4×4→5×5→6×6→7×7
 田块	产量	0→3	2500/5000/10000	产量 +0%→+20%→+40%→+50%

🕒	田块	生长速度	0→3	2500/5000/10000	时间	×1.0→×0.9→×0.8→×0.7
---	----	------	-----	-----------------	----	---------------------

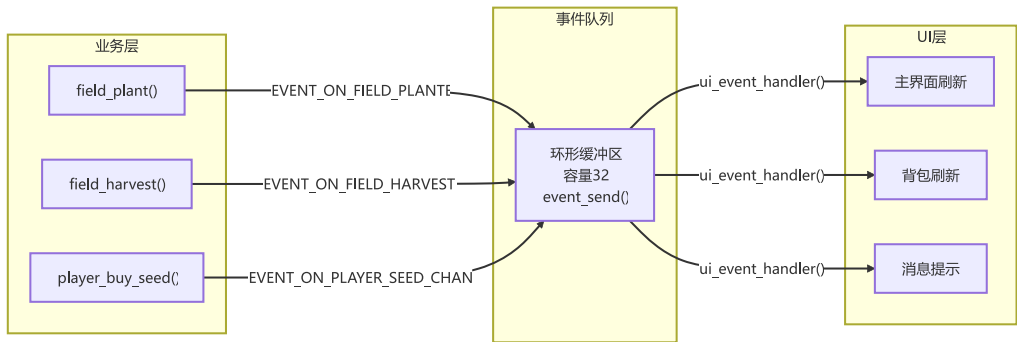
🌱	田块	耐虫性	0→3	10000/20000/50000	抗性	+0→+0.1→+0.25→+0.4
---	----	-----	-----	-------------------	----	--------------------

其中田块升级采用"下一季生效"策略——升级不影响当季作物，收获后下一季种植自动应用。

3.4 事件系统（event）

事件系统是本项目的核心设计亮点之一，通过环形队列实现业务逻辑层与UI 层的完全解耦。

3.4.1 设计原理



共定义 17 种事件类型，覆盖种植、收获、虫害、升级、金币/经验变化、无人机状态切换等所有关键状态变更。事件队列容量为 32，采用覆盖旧事件的策略防止队列溢出。

3.4.2 实现细节

```
#define EVENT_QUEUE_SIZE 32

void event_send(event_type_t type, void *data) {
    event_sequence[event_tail].type = type;
    event_sequence[event_tail].data = data;
    event_tail = (event_tail + 1U) % EVENT_QUEUE_SIZE;
    if (event_count >= EVENT_QUEUE_SIZE) {
        event_head = (event_head + 1U) % EVENT_QUEUE_SIZE; // 覆盖旧事件
    } else {
        event_count++;
    }
}
```

主循环中一次性消费所有积压事件，避免事件堆积：

```
while (1) {
    speaker_update();
    lv_timer_handler();
    event_t *event;
    while ((event = event_get()) != NULL) {
        ui_event_handler(event); // 一次性处理所有积压事件
    }
    delay_us(2000);
}
```

3.5 UI 系统设计

3.5.1 整体布局

UI 基于 LVGL v8.2 构建，采用单屏幕多窗口的设计模式：

- 加载界面 (ui_load)：启动画面，显示游戏标题和加载按钮。
- 主界面 (ui_main)：核心游戏界面，包含农田网格、顶栏（金币/EXP/等级）、周围功能按钮、各种弹出窗口。

游戏主要布局截图如下：



加载页



主界面



无人机操作面板



拖动种植窗口



商店窗口



仓库窗口



无人机工作状态 HUD



设置窗口

3.5.2 窗口管理体系

弹出窗口（商店、仓库、设置、种植、农田升级）采用统一的创建/销毁管理机制，通过 `ui_window_toggle_desc_t` 结构描述窗口的创建函数与实例引用，实现一键切换开关：

```
typedef struct {
    lv_obj_t *(*create)(void);    // 延迟创建函数
    lv_obj_t **window_ref;       // 窗口实例指针
} ui_window_toggle_desc_t;
```

窗口通过统一的组件库 `ui/widgets/ui_window.c/h` 创建，可设置有无遮罩、跟随滚动、是否关闭后销毁等。

3.5.3 消息提示

通过 `ui/widgets/ui_message.c/h` 组件发送顶部消息提示，消息有 `toast` 与 `confirm` 两种类型，支持多种风格。

消息提示涵盖操作成功与否、无人机状态变更、作物濒死告警等场景，让玩家能清晰地了解游戏状态，提升游戏体验。

```
#define UI_MESSAGE_MAX 5 // 最多同时显示的消息数量
#define UI_MESSAGE_TOAST_DURATION 2500 // Toast 消息显示时长（毫秒）

/* 消息样式 */
typedef enum {
    UI_MESSAGE_TYPE_INFO, // 一般信息，灰色风格
    UI_MESSAGE_TYPE_WARNING, // 警告信息，黄色风格
    UI_MESSAGE_TYPE_SUCCESS, // 成功信息，蓝色风格
    UI_MESSAGE_TYPE_ERROR, // 错误信息，红色风格
} ui_message_style_t;

/* 消息类型 */
typedef enum {
    UI_MESSAGE_TOAST, // 短暂显示后自动消失
    UI_MESSAGE_CONFIRM, // 需要用户确认
} ui_message_type_t;
```

3.5.4 刷新机制

UI 刷新采用双定时器 + 事件驱动策略：

- **100ms 定时器**：刷新无人机位置、HUD 面板数据等需要实时更新的数据。
- **1s 定时器**：刷新农田网格（主要是生长时间/死亡进度条的刷新），与生长时间刻对齐。

事件驱动的刷新仅在状态变更时触发（如金币/等级、作物贴图更换等），避免无效刷新，节省开销。

大部分刷新通过事件驱动。

3.6 数据持久化（存档系统）

3.6.1 存档策略

基于 SD 卡 + FATFS 文件系统实现，三个核心模块各自独立存档：

存档文件	内容
0:/farm_save.dat	farm 对象 ：农场规模、所有田块完整状态（坐标/作物/生长进度/虫害/升级等级/产量因子等）
0:/player_save.dat	player 对象 ：金币/经验/等级/背包（种子/收获/农药）

0:/drone_save.dat

drone 对象：无人机位置/状态/速度/药仓/算法等级/检测矩阵

存档在心跳定时器中每秒自动执行一次。系统启动时自动检测存档文件，存在则恢复，不存在则创建新档。

3.6.2 数据序列化

根据各模块数据结构的特点，采用了两种不同的序列化策略：

（1）逐字段序列化（farm 模块）：由于 farm_t 内部包含指向堆分配 field_t 的指针，无法直接整体写入，因此通过 save_utils.h 中封装的辅助函数逐字段读写：

```
static inline bool save_write_int(FIL *fil, int value);
static inline bool save_read_int(FIL *fil, int *value);
static inline bool save_write_double(FIL *fil, double value);
static inline bool save_read_double(FIL *fil, double *value);
#define SAVE_CHECK(expr) do { if (!(expr)) { f_close(&fil); return false; } } while (0)
```

每个字段的读写均通过 SAVE_CHECK 宏校验，失败时自动关闭文件并返回错误。存档时按"先写农场头部（规模/等级），再嵌套循环逐个写入田块的全部状态字段"的顺序组织数据。

（2）整体结构体读写（player / drone 模块）：由于 player_t 和 drone_t 均为扁平结构体（所有数据内联，不含指针），可直接将整个结构体作为二进制块写入/读取，代码简洁高效：

```
// drone 存档（player 同理）
if (f_write(&fil, s_drone, sizeof(drone_t), &bw) != FR_OK || bw != sizeof(drone_t)) {
    f_close(&fil);
    return false;
}
```

两种策略的选择原则是：含指针或嵌套堆对象的结构体采用逐字段序列化，扁平结构体采用整体二进制读写，在安全性与效率之间取得平衡。

3.7 硬件驱动适配

3.7.1 真随机数

利用 GD32H7 内置 TRNG（真随机数生成器）硬件模块产生随机种子，用于虫害感染概率判断，确保每次游戏的虫害事件不可预测：

```
trng_mode_config(TRNG_MODSEL_NIST);
trng_clockerror_detection_enable();
trng_enable();
while (trng_flag_get(TRNG_FLAG_DRDY) == RESET);
srand(trng_get_true_random_data());
```

3.7.2 扬声器音频流

音频基于 SAI 接口 +DMA 双缓冲实现流式输出。在主循环中每次 LVGL 渲染前优先补充音频缓冲 speaker_update(), 环形缓冲（16384 采样 $\approx 372\text{ms}$ ）足够覆盖任一帧的渲染耗时，避免渲染阻塞导致音频欠载。无 SD 卡时音频自动静默，不影响游戏运行。

3.7.3 摇杆输入

摇杆驱动封装在 joystick.c，提供方向向量接口供无人机移动使用，支持连续读取。

四、 课题设计结果

4.1 最终实现的功能

经过完整的开发周期，本系统实现了以下全部功能：

（1）农场作物全生命周期管理

- ☒ 三种作物（小麦/水稻/玉米）的种植、六阶段生长、成熟收获全流程。
- ☒ 支持通过拖拽进行种植。
- ☒ 支持单个收获与一键收获两种收获方式。
- ☒ $n \times n$ 农田网格与作物生长时间可视化展示，作物各阶段、虫害各种类型有不同图标。

(2) 病虫害系统

- ☒ 四种虫害（蚜虫/螨虫/卷叶蛾/蝗虫），基于二次函数概率感染模型。
- ☒ 虫害持续减产直至作物死亡，死亡后田块自动清空。
- ☒ 虫害类型在无人机检测前对玩家隐藏，检测后展示对应虫害图标。

(3) 无人机防治系统

- ☒ 三种工作状态（空闲/侦查/喷洒）完整切换。
- ☒ 侦查模式下手动通过摇杆控制飞行，检测虫害，HUD 面板实时显示检测到的虫害统计。
- ☒ 喷洒路径采用贪心+2-opt 优化算法自动规划，兼顾路径质量与计算效率。
- ☒ 药仓管理：装载农药→自动匹配喷洒→药量不足提示补药。

(4) 经济与成长系统

- ☒ 商店购买种子和农药，仓库售卖收获作物。
- ☒ 金币实时结算，经验累积自动升级。
- ☒ 6 阶段 40 级等级系统，对应 7 级折扣率（7 折~原价）。
- ☒ 6 大升级体系完整实现（无人机×2、农场×1、田块×3）。

(5) 数据持久化

- ☒ 每秒自动存档到 SD 卡，断电重启无缝恢复。
- ☒ 无 SD 卡时降级正常运行（不存档、无音效）。

(6) UI 与交互

- ☒ 基于 LVGL 的完整图形界面，包含加载页、主界面、商店/仓库/设置/种植/升级等窗口。
- ☒ 摇杆操控无人机飞行，触控点击操作 UI 控件。
- ☒ 背景音乐与操作音效。
- ☒ 消息提示系统（购买/收获/濒死等事件弹出提示）。

4.2 工程代码统计

类别	有效代码行数	占比
核心业务逻辑模块（modules/）	1,245 行	23.3%
UI 布局实现（ui/layouts/）	2,421 行	45.2%
UI 回调函数（ui/callbacks/）	570 行	10.6%
UI 控件库（ui/widgets/）	666 行	12.4%
UI 主模块与公共工具（ui/.）	387 行	7.2%
核心调度层（main.c）	69 行	1.3%
总计	5358 行	100%

4.3 系统运行效果

- 游戏上电后进入加载页面，按下启动键进入主界面。
- 画面刷新流畅，农田网格、作物图标、虫害标记渲染正确无花屏。
- 摇杆操控无人机响应及时，飞行路径平滑。
- 音效播放流畅连贯，无卡顿、噪音。
- 所有窗口切换动画流畅，触控操作灵敏。
- SD 卡存档自动进行，掉电后重新上电进度恢复无误。
- 长时间运行无死机、内存泄漏现象。

五、 总结与展望

5.1 核心工作内容与成果

本项目成功在 GD32H7 嵌入式平台上实现了一款功能完整、交互流畅的农场经营模拟游戏。主要成果包括：

1. **清晰的分层架构设计：**硬件驱动层→核心调度层→业务逻辑层→事件通信层→UI 表现层的五层架构，模块边界清晰，层级/模块之间低耦合，代码可维护性强。
2. **核心玩法完整实现：**种植→生长→虫害→侦查→喷洒→收获→售卖→升级的完整循环全部实现，玩法深度达到设计预期。

3. **事件驱动架构**：通过事件队列实现业务逻辑与 UI 的完全解耦，有效降低模块耦合，这是本游戏最重要的架构实践。
4. **嵌入式性能优化**：UI 采用双定时器刷新策略（100ms+1s）+ 事件驱动按需刷新，在资源受限的单片机上实现了流畅的交互体验。
5. **数据持久化**：基于 FATFS 实现了完整的存档系统，支持掉电恢复，并支持无 SD 卡正常运行。
6. **算法实践**：在无人机路径规划中实现了贪心+2-opt 优化的组合算法，将所学的算法知识应用于实际工程。

最终交付代码量约 8,300 行（不含工程模板、硬件驱动与 C 数组图标文件），涵盖 6 个核心业务模块、7 个 UI 页面布局、3 个可复用控件等。

5.2 设计核心要点

1. **单例模式**：farm、player、drone 均采用静态存储+单例访问，避免全局变量扩散，同时支持存档恢复时的静态绑定。
2. **模块化开发**：逻辑层与 UI 层分离，逻辑层内部分为六个模块，按面向对象思想开发；UI 层将布局与回调分离，内部又按不同界面分离。结构清晰，可维护性强。
3. **配置与逻辑分离**：所有游戏参数（价格、经验、升级参数、虫害模型）集中在 data.c 中，调整游戏平衡只需修改配置数组，无需改动逻辑代码。
4. **事件机制**：通过逻辑层发送事件通知 UI 层状态变化，UI 层消费事件进行 UI 刷新，解耦 UI 层与逻辑层。

5.3 存在的不足与优化思路

1. **事件队列容量固定**：当前容量为 32，在极端高频操作下可能出现旧事件被覆盖的风险。优化思路：动态扩容或对同类事件合并（如多次金币变化合并为一次）。
2. **存档策略较为粗暴**：目前采用全量序列化，每次存档写入所有数据。优化思路：增量存档，仅写入变更字段，减少 SD 卡写入量。

3. **功能方面可以进一步拓展：**可以引入更复杂的作物生长因素，如拓展浇水、施肥机制，引入天气系统；增设任务系统、成就系统；增加新手指引；引入更丰富的作物类型和虫害类型（在此基础上还可加入作物解锁机制）；实现多账号共存与切换等。
4. **部分 UI 可以进一步完善：**如作物成熟或死亡可以增加单独的 UI 视觉效果，作物收获、无人机喷洒农药等操作可以增加动画效果，游戏界面可以增加更多装饰性农场元素，游戏背景、UI 风格可以进一步美化，游戏可以增加多语言支持。

5.4 未来展望

1. **更多作物与虫害类型：**扩展作物种类（如大豆、棉花等）和虫害种类，增加解锁机制，丰富玩法策略深度。
2. **天气系统：**引入晴天/阴天/雨天等天气因素，影响作物生长速度和虫害发生概率。
3. **排行榜与成就系统：**在 SD 卡上存储历史最高分和成就解锁状态，增加游戏重玩价值。
4. **多语言支持：**中英文界面切换。

最后更新于：2026 年 6 月 5 日

附录：课题分工说明

分工情况				
成员	姓名	主要分工	有效 代码量	工作量 占比
I	刘俊辉	UI 交互系统设计与实现（主界面/商店/仓库/设置/无人机/升级等全部页面布局与回调）；窗口管理与消息系统；样式、音效与动画效果；硬件驱动与 UI 的对接（摇杆/扬声器）	4044 行	50%
II	吴天宇	核心业务逻辑模块设计与实现（farm/field/player/drone/event/data），包括农场田块管理、作物生长与虫害模型、无人机状态机与路径规划算法、玩家经济与等级系统、事件通信系统、FATFS 存档读写等；主循环调度；游戏配置数据设计；图片与音效资源搜集	1314 行	50%

工程代码统计		
类别	有效代码行数	占比
核心业务逻辑模块（modules/）	1,245 行	23.3%
UI 布局实现（ui/layouts/）	2,421 行	45.2%
UI 回调函数（ui/callbacks/）	570 行	10.6%
UI 控件库（ui/widgets/）	666 行	12.4%
UI 主模块与公共工具（ui/.）	387 行	7.2%
核心调度层（main.c）	69 行	1.3%
总计	5358 行	100%